

AIWA Yellow Paper

Causal Coordination and Local Value Accrual for Partition-Tolerant Networks

Version 1.1 — reference implementation specification

Abstract

AIWA is a distributed coordination and value-accrual primitive for networks operating under arbitrary communication delay, intermittent connectivity, and long-duration partition.

AIWA separates two problems that are conventionally coupled.

Causal coordination derives a deterministic causal layer for authenticated events from a common genesis, causal history, and Mirror observations. Independent domains continue operating while disconnected and reconcile later, when evidence becomes available.

Accrual provides controlled, *local* creation of value. An identity is activated through an externally anchored Genesis Commitment (a real Solana burn). Its claimable value then evolves according to a deterministic progression function, driven by real sequential computation, not a shared clock. Claiming converts accrued value into native AIWA balance and resets the domain's own patience clock. Once claimed, AIWA transfers between wallets through authenticated causal history, with no continued dependency on the external activation chain.

Progression determines how much value a domain may claim.

Conservation determines who owns the resulting value.

Mirror determines what history can be verified.

Neither requires a globally synchronized state.

1 System model

An AIWA network consists of autonomous identities and domains. An identity is controlled by a cryptographic key pair. A domain is an operational environment holding one or more identities and their local state.

Communication may be continuous, intermittent, delayed, asymmetric, or unavailable for an arbitrary period.

AIWA assumes standard cryptographic primitives — collision-resistant hashing, secure digital signatures, deterministic serialization — and does not require a globally synchronized wall clock, continuous external-chain access, a global consensus quorum, or a globally replicated balance state.

2 Identity

An identity is a public key. Its address is:

$$\text{domain}(i) = \text{SHA-256}(\text{publicKey}_i)$$

the full 256-bit digest, hex-encoded, never truncated. The private key authorizes actions; the physical device is not the identity. An identity can move between devices — Earth, a spacecraft, Mars, a new device — while retaining the same cryptographic identity, with no inherent binding to a location, IP address, or machine.

3 Event DAG

All state transitions are signed events referencing their causal predecessors, forming a directed acyclic graph. No participant is required to possess every event; a participant maintains and verifies the history relevant to its own state and the relationships it observes. Merging two independently-grown DAGs is a pure, commutative union.

4 Mirror

A Mirror is a domain’s local, signed, authenticated record of what it has actually observed and verified concerning other domains — never a replica of the observed domain, never a global state database.

A commitment is either **empty** (nothing received this epoch) or **full** (a real, verifiable set of references to source events). Reception monotonicity holds: a domain’s own successive claims about what it has seen of another domain must never claim to have seen *less* than it previously, verifiably claimed.

Mirror does not prove two domains are distinct real entities, and does not rule out a genuinely collaborating pair fabricating a consistent history together — no purely relational mechanism, with no external anchor, can close that gap. It is real evidence about the structure of observed history, not an identity oracle.

5 Progression

A domain’s own progression epoch advances only through a valid transition: exactly +1, causally chained to the domain’s own last accepted transition, and carrying a real sequential proof (§6).

Progression is never shared or synchronized across domains. Two domains’ epoch counts are never directly comparable — each is a purely local measure of that domain’s own advancement.

6 The sequential proof

Each progression transition carries a real, verifiable sequential hash chain:

$$h_0 = \text{SHA-256}(\text{seed}), \quad h_i = \text{SHA-256}(h_{i-1})$$

seeded from the domain and the previous epoch’s own output — chaining across epochs, not merely within one, so epoch N cannot begin before epoch $N - 1$ has genuinely finished.

This computation is always local to a single, real device — never distributed across the network, never something other domains help compute. Whatever device currently holds and runs a domain’s own keypair is the one real device advancing that domain’s own next epoch, right now; nothing else ever contributes to it. The network (Mirror, §4; live sync) only ever carries already-computed, already-verified results afterward — never the computation itself, and never any part of it. Since identity is the keypair, not the device (§13.1’s own point), this real device can change over time — moving to a different, real machine restores the identical real identity and its full, already-verified history there, and that new machine becomes the one computing

whatever comes next; nothing about the domain’s own past or claimable value depends on which real device that ever was or will be.

This is not an asymmetric cryptographic VDF (Wesolowski, Pietrzak) — verifying costs exactly what producing costs, not asymptotically less. What it provides is unconditional: production cannot be parallelized or shortcut by any amount of hardware. It bounds the *rate* at which a domain can advance its own progression; it makes no claim about calendar time. A domain with sustained, continuous compute available genuinely can advance faster than one without it — that is the property, not a flaw in it.

Precisely because each transition is genuinely sequential, one epoch is a real mirror of real, physical elapsed time — measured, not assumed: a real 12,000-iteration chain (this deployment’s own `VDF_ITERATIONS`) takes a real, measured $\sim 256\text{ms}$ of genuine wall-clock computation. No domain ever consults a clock to know this; the real, physical time simply cannot be skipped. But this mirror is a floor, not a symmetric measurement between domains: nothing stops a domain from taking longer per epoch (weaker hardware), and two domains’ own real hardware for this specific computation may genuinely differ. Epoch count is therefore a real, honest measure of how much genuine sequential work *this* domain has done — correlated with, but never identical to, how much real time has elapsed for some *other* domain. This is exactly why raw epoch counts across domains are never directly compared (§5’s own point) — not because the physical-time mirror isn’t real, but because it is calibrated differently per domain’s own real hardware, and adjusting one domain’s own earned value by comparing it to another’s would either discount real, honest work (a slower domain penalized for real hardware limits) or reward idle time (a domain credited for simply reconnecting) — see §13’s own discussion of why Causal Tick stays informational, never a correction to what has actually been earned.

7 Accrual

Once activated (§8), a domain accrues claimable AIWA as a function of three quantities:

$$R(S, t, A) = \frac{S \cdot t^\alpha}{\left[\beta \ln A + \ln \left(1 + \frac{C}{A^\beta} \right) \right]^\gamma}$$

- S — committed value (from Genesis Commitment and any later commitment).
- t — real progression epochs elapsed **since this domain’s own last economic action** (a burn or a claim). Resets to zero on every such action.
- A — this domain’s own **total** progression age. Never resets.
- α, β, γ, C — deployment parameters.

7.1 Why t resets and A does not

This is the one precise, load-bearing distinction in the whole formula, and an earlier draft of this specification did not state it clearly enough to prevent a real implementation bug: t must never be a value the claiming domain supplies. A domain that could declare its own reference epoch could always declare the earliest one available, maximizing t forever. t and A are therefore both derived exclusively from the domain’s own already-verified progression state (§5) at the moment of the query — recomputed, never trusted from any event payload.

Resetting t on every claim gives the formula its real economic meaning: reward is for patience since you last touched your position, not since you first opened it. A , left unreset, provides the formula’s own denominator — a measure of overall maturity independent of any individual domain’s own claiming pattern.

7.2 The computable form

The denominator is represented as $\beta \ln A + \ln(1 + C/A^\beta)$ rather than $\ln(A^\beta + C)$ directly — the same value ($\beta \ln A + \ln(1 + C/A^\beta) = \ln(A^\beta + C)$, a real logarithmic identity), computed this way to reduce overflow risk for large A .

8 Genesis Commitment

Identity activation is a real, irreversible SOL burn to Solana’s public incinerator address, verified from an already-fetched, finalized transaction record — never a self-declared amount. The same keypair that burns is the keypair AIWA accrues to; there is no second, AIWA-only key.

This one step is the real exception to everything else in this document: broadcasting a real burn, and any other real interaction with Solana itself (checking a real balance, sending real SOL), requires reaching a real, centralized, Earth-hosted RPC endpoint over real internet — the identical real dependency any Solana wallet has. Once activated, a domain’s own AIWA progression (§5–§7) needs no further contact with Solana, or with Earth, ever again — a domain can activate on Earth, then travel, then progress locally, uninterrupted, for any real duration. But activation itself, and any further real Solana-side action, cannot happen from somewhere with no real path back to Earth’s own network.

A domain’s real, total committed capital accumulates across every valid burn it makes, not just the first — matching accrual’s own b (§7), which already accumulates identically. An earlier version rejected any burn after a domain’s first outright, silently understating a domain’s real weight in Causal Tick (§13) even as its own accrual capital kept growing from the identical real burns; a real, closed inconsistency, not a hypothetical one. Each individual burn is still checked against the deployment’s own churn cost curve, if any, at its own real slot — a later commitment must meet whatever the curve currently requires, never grandfathered in at an earlier, possibly lower, requirement.

The accrual formula is linear in S — absent an identity cost, splitting capital across many identities would not by itself reduce total accrual. A per-identity activation cost, optionally scaled by a real cost curve as protocol time passes, makes churn (abandoning an aged domain for a fresh one) strictly costlier, not free. A new identity inherits none of an old one’s progression, Mirror history, or claimed value. Whether a given deployment’s real cost curve actually makes churn net-unprofitable — not merely costly — is a genuine, parameter-dependent economic question; `public/core/churn-analysis.js` computes the real, honest answer for a concrete choice of parameters rather than asserting it in general.

9 Conservation

Once claimed, AIWA is a discrete, owned unit (a claim), moved by a real, signed transfer, or divided by a real, signed split into two claims whose amounts sum exactly to the original — enforced by construction (exact integer subtraction), not by a separate check that could be wrong.

A transfer requires a real signature proving control over the source domain; an unsigned or forged transfer is rejected before it can touch any claim. Neither operation requires continued access to the domain’s own progression or to the external activation chain — a claimed AIWA is fully portable local value.

10 Denomination

AIWA uses 18 decimal places. $1 \text{ AIWA} = 10^{18}$ base units. Every claim, transfer, and split amount is a real integer count of base units — never a floating-point approximation of a balance. The accrual formula’s own fractional exponents require floating-point computation internally; the result is converted to an exact integer at the one real boundary where it becomes part of a balance, faithful to what the float actually holds.

11 Partition and reconciliation

During a partition, each domain continues independently — no domain decrements its own state because another is unreachable, and none waits for permission to continue. Reconciliation, when connectivity resumes, is real signature verification, real ancestry verification, and real Mirror-observation verification over the newly available evidence — never a question of whose clock is correct.

11.1 Value creation, not only value transfer, under real latency

Existing published work on interplanetary cryptocurrency — for example, real, recent academic proposals combining Bitcoin with Delay/Disruption-Tolerant Networking (DTN, itself a real, NASA-originated field for deep-space communication) — generally extends a single, Earth-anchored consensus chain across the latency gap: delay-tolerant transport, payment-channel timelocks widened to real light-time delay, and federated or merge-mined settlement, while leaving that chain’s own consensus and monetary issuance untouched. This is a real, workable answer to transferring already-created value under latency. It carries a real, acknowledged limitation on the other real half of the problem: value *creation* stays structurally dominated by whichever side has more real compute — the same literature’s own account is that mining from Mars, competing against Earth’s own hashpower, would be unprofitable by design.

This project’s own real design choice differs in kind, not merely degree: value creation (§5–§7) never requires reaching, or even being aware of, any consensus chain. A domain on Mars creates real, VDF-bound value at its own real pace, from its own real committed capital, with zero dependency on Earth’s own compute, consensus, or connectivity. Reconciliation (§4, §13) happens only afterward, and only ever adds a real, weighted, informational signal — it never revises, discounts, or waits on agreement about what has already been genuinely created.

This closes the specific “Mars cannot fairly create value” gap the extend-a-single-chain approach carries by its own design. It does not solve, and does not attempt to solve, what that approach is actually built to solve: reliably moving real bytes across a real, high-latency, intermittent link — this project’s own `p2p-connection.js` reuses the general, real DTN idea (store real data locally, forward it when a real link exists) rather than reinventing it, and remains real but unautomatedly tested (see the README’s own honest limits). Nor does it establish any real exchange rate between two economies that grew up separately, with no shared reference, and later reconnect — a real, open economic question, outside this project’s own scope.

12 Explicit non-claims

AIWA does not claim to solve the human-identity oracle, perfect physical-location verification, absolute global time, Byzantine agreement without assumptions, or detection of every coalition of identities controlled by one real actor. These are structural boundaries, stated directly, not omitted implementation details.

A coalition can produce internally consistent history at real cost — valid keys, valid signatures, valid ancestry, valid Mirror commitments, and whatever economic commitment the protocol requires. AIWA’s claim is narrower and precise: fabricated identities cannot fabricate authenticated history for free.

13 Causal Tick

A domain’s own progression (§5) remains the unconditional source of truth for its own accrual — real, VDF-bound, secure even with zero external observers. A second, complementary mechanism provides what local progression alone cannot: a shared, externally-corroborated position, useful for reconciliation and for detecting a self-reported epoch that has drifted implausibly far from what independent observation supports.

For a target domain, every other domain’s Mirror commitments referencing it are gathered, each weighted by that observer’s own real, already-verified Genesis Commitment burn (§8) — never a self-declared score. Each real observer contributes exactly one estimate — their own most-recent, highest-resolving observation of the target — never one estimate per historical commitment referencing the same fact. An earlier version of this counted a replayed or repeatedly-referenced observation once per mention, letting a single real observer’s weight be counted more than once without any additional real evidence; a real, closed gap, not a hypothetical one. The weighted median of these single, current estimates is the Causal Tick; the real spread between the lowest and highest individual observation is reported alongside it as an interval, not discarded. A minority of adversarial weight cannot pull the result arbitrarily, as long as it stays below half of the total real weight observing that domain — the same real, well-studied security shape as proof-of-stake, applied here to causal positioning.

A domain with no external observers yet has no Causal Tick at all ($\perp$, insufficiently determined) — a real, honest absence, never treated as inconsistency.

Two domains with entirely unrelated local progression — different total elapsed epochs, different committed capital, no synchronization with each other at all — compute the identical Causal Tick for the same third, externally-observed domain, given the same real evidence. This holds for free from event ids already being content-addressed (§3): the identical causal content produces the identical id regardless of how many hops or which path it traveled through, so two domains that come to know about the same event by different routes are, from the protocol’s own point of view, looking at the exact same fact. A real, structural measure of “how many distinct branches of new causal content arrived” was considered and rejected as the basis for this — it cannot be distinguished from a well-resourced actor fabricating many genuinely distinct (never literally replayed) branches, the identical gap this section’s own real committed-capital weighting already closes.

The Causal Tick never adjusts what a domain has actually, verifiably earned — only what is reported alongside it. Actively bounding a domain’s own real accrual by comparison with corroborating peers was considered directly: reducing a slower domain’s real earned value to match a faster one’s would punish real, honest work done under real, weaker hardware (§6’s own point); inflating a reconnecting domain’s value toward a peer’s would create value from elapsed idle time rather than real sequential computation, and would be directly exploitable by deliberately isolating and reconnecting. Neither direction is implemented; `checkCausalConsistency`’s own real gap is reported, never corrected.

13.1 Hardware roots are optional, and never an authority

AIWA’s evidence interface works with software primitives alone. A domain may optionally strengthen its own independence assurance with real, physically-provisioned hardware roots — but hardware never computes the Causal Tick, never sets or scales any observer’s weight, and never becomes a required input. A domain backed by zero hardware attestations and one backed by several produce an identical Causal Tick from identical Mirror evidence; only a separate, informational confidence signal (how much of the corroborating weight is also hardware-backed) differs.

A real attestation is a two-hop signature chain: an already-established origin domain signs the issuance of a specific hardware root; that hardware root’s own key signs its binding to a specific domain. Both signatures are independently verified — never trusted from the record’s own claims about itself. A real minimum of two distinct, independently-issued roots is required for a domain’s own minimal independence hypothesis to hold; one root only moves trust to a single unit, never establishes a real minimum.

Stated directly, the one honest limit no cryptographic protocol closes: nothing here can verify from software alone that a hardware root’s key really lives on real, non-clonable hardware. What real verification provides is a real, traceable chain of signatures from a known origin — reducing the trust surface from “any of many sybil identities” to “compromising the origin’s own issuance process, or one specific physical unit” — a real, meaningful reduction, never an absolute physical guarantee.

Identity is the keypair, never a specific piece of hardware — losing a device, or a hardware root, is real, but never identity loss: the same real domain is restored, unchanged, from a saved key or a passphrase (§8’s own `solana-wallet.js`) on any other real, reachable device. This closes a narrower, real problem — a single lost or destroyed device — but not the one §11 itself is built around: a domain with genuinely no reachable device or channel at all, for a real, extended period (an actual communication blackout, not a broken laptop). In that case no device, old or new, has anything to reconnect to, regardless of which key it holds — hardware roots strengthen independence assurance for a domain that IS reachable; they do not, and are not meant to, substitute for reachability itself during a real partition.

Reference implementation

Concept	File
Identity	public/core/domain-id.js
Event DAG	public/core/event-dag.js
Sequential proof (§6)	public/core/vdf.js, public/core/wesolowski-vdf.js (asymmetric, real cheap verification), public/core/bigint-math.js
Progression (§5)	public/core/progression.js
Accrual formula (§7)	public/core/reward.js
Accrual position, t/A (§7.1)	public/core/accrual.js
Genesis Commitment (§8)	public/core/identity-cost.js, public/core/solana-wallet.js
Churn profitability check (§8)	public/core/churn-analysis.js — a real, parameter- specific answer, never a general guarantee
Cross-runtime interoperability	interop/rust-vdf/ — a real, independent Rust implemen- tation, verified byte-for-byte identical to the real JS output
Conservation (§9)	public/core/conservation.js
Denomination (§10)	public/core/units.js
Mirror (§4)	public/core/mirror.js
Causal Tick (§13)	public/core/causal-tick.js, public/core/weighted-median.js
Hardware roots, optional (§13.1)	public/core/hardware-attestation.js
Live peer-to-peer sync	public/core/p2p-signaling.js, public/app/p2p-connection.js
Coherent composition	public/core/wallet.js

229 real tests cover this table — every one but the real cross-runtime check runs unconditionally; that one skips gracefully, never fails, without a real Rust toolchain available. Security-relevant cases are named as such in their own test files.